

Implementation

```
import Data.Bits
import Data.List
import Data.List.Elem
import Data.List1
import Data.Vect
import Data.List.Quantifiers
import System
```

Defining the valid C types

Idris has builtin support for some of the types that naturally translate to C:

- fixed, signed and unsigned integers
- Double precision floating point numbers

The following description only considers `Int` for brevity. Additionally, an opaque `Float` needs to be defined for descriptive purposes, but it is marshalled from and unmarshalled to `Double` on the Idris side (more on this later).

```
data Float : Type where
```

TODO: `Ptr`

Next, compound types `Struct` and `Array` are defined using dependent types. A `CArray` is defined by an integer number of elements and the type of those elements (footnote: the type is named `CArray` because `Array` is the canonical name for a similar but pure-idris concept and the back-end handling of FFI-crossable types depends only on the name of the type; support for this has not made it to the mainline compiler):

```
data CArray : Int -> Type -> Type where
```

A `Struct` is defined by a string name and a list of fields which consist of the string names and types:

```
-- This type already exists in System.FFI
data Struct : String -> List (String, Type) -> Type where [external]
```

Note that the type of a field can be anything, not just a C type (eg. a function, a dependent type, etc.). In order to limit which types are supported C types an indexed data type can be used:

```
data CType : Type -> Type where
```

This will have a couple interpretations:

- `CType a` is the universe of types that I've defined are valid in both Idris and C
- a `CType a` is a proof that some Idris type `a` is representable in C

The former is important because it highlights the subjectivity of these choices and that that this is “just another type” in the eyes of the type checker. Meaning that, as will be demonstrated, no special-casing or hardcoding of these types within the compiler is needed for (de)allocation, reading, or writing to their values. (footnote: the sizes of these types still needs to be hardcoded and made accessible somewhere but this is a much simpler task than eg. normalizing and evaluating `CType` within the compiler <https://idris2.readthedocs.io/en/latest/backends/backend-cookbook.html#foreign-types>) (footnote: it may be needed for building FFI calls; TODO).

The latter interpretation reflects the general usage of `CType` within the codebase, and provides a better plain-English understanding of `CType`.

Starting from the basics, the constructors for `CType` are:

```

CInt : CType Int
CFloat : CType Float
CDouble : CType Double

```

CInt is a proof that Int has a C representation, CFloat is a proof that Float has a C representation, and CDouble is a proof that Double has a C representation, etc. The utility of this proof can be seen in the following CType:

```

CPtr : CType t -> CType (Ptr t)

```

CPtr uses CType recursively to say “given a type t with a C representation, Ptr t has a C representation. This is used similarly for arrays, with the additional Int needed for the array’s length:

```

CCArray : (n : Int) -> CType t -> CType (CArray n t)

```

Struct looks more complicated, but follows the same principle.

```

-- the versions with the . operator plays nicer with the typechecker
-- but the meaning is the same as
-- CStruct : All (\(fname, t) => CType t) fields -> CType (Struct name fields)
CStruct : All (CType . Builtin.snd) fields -> CType (Struct name fields)

```

Rather than a single other CType t, all of a Struct’s fields need to have C representations. This is done using a Data.List.Quantifiers.All. In mathematical language, the type All p l is a list of different propositions and a value of such type is a list of proofs for those propositions. (TODO: note about this exact formulation of all?) Thus, in CStruct, given proofs that all the types of a struct’s fields is representable in C, then the struct itself is representable in C.

Allocation

Allocation’s require some support from backend, specifically calling the standard library function malloc(3).

```

%foreign "scheme:foreign-alloc"
prim__malloc : Int64 -> PrimIO AnyPtr

```

This simply allocates a specified amount of memory, but we’d like something more in the shape of:

```

alloc : (repr : CType a) -> IO (Ptr a)

```

Going from PrimIO AnyPtr to IO (Ptr a) is matter of calling primIO and prim__castPtr; so what’s needed is CType a -> Int, attentive readers may recognize this as a familiar compiler builtin:

```

sizeof : (repr : CType a) -> Int64

```

Assuming modern 64-bit system and normal compiler (footnote: see above footnote about hardcoding values, one potential solution is providing pragmas for non-portably sized types int, char, pointers, etc.), implementing this should be straightforward for most CTypes. The one exception is CStruct, where alignment and padding need to be considered. The details are out of scope here, but the short version is that values need to be on integer multiples of their sizes, and this alignment needs to be preserved within arrays. Thus, alignof is also needed.

```

alignof : (repr : CType a) -> Int64

```

Finally:

```

alloc repr = map prim__castPtr $ primIO $ prim__malloc (sizeof repr)

```

And for completeness, free is trivial

```
%foreign "scheme:foreign-free"
prim__free : AnyPtr -> PrimIO ()

free : Ptr a -> IO ()
free ptr = primIO $ prim__free $ prim__forgetPtr ptr
```

Memory safety with lifetimes

Now that memory can be allocated, the type system can be used to make sure this is done safely and correctly. Lifetimes, as described herein, avoid a form of unsafe behavior known as “use-after-free”. Use-after-free bugs are a common cause of security vulnerabilities (TODO: citation needed). Additionally the scope pattern avoids memory leaks, that is forgetting to release memory back to the allocator such that the program consumes more and more memory which could be recycled. This is not necessarily a safety problem, but an equally difficult problem to address.

Another unsafe behavior, accessing uninitialized memory, is addressed by `Marshal` in the following section.

—

After allocation and initialization, memory that is no longer used needs to be deallocated and returned to the operating system with the `free(3)` system call. Further, after the call to `free` the pointer should not be read from or written to, doing so is known as “use-after-free” (footnote: a future call to `malloc` can return a pointer to the same address, but whether the original and new pointers are considered the same depends on the memory model; a related issue is pointer provenance)

The design takes inspiration from Rust where “references” are pointers with an associated lifetime. Rust’s lifetimes are a special construct within the type system with different behavior and usage from normal datatypes. In this library, `Lifetimes` are modeled as a plain data type; and the `Ref` type thus a dependent type with a `Lifetime` value as well as the pointed-to `Type`.

```
-- constructors defined later
data Lifetime : Type

data Ref : Lifetime -> Type -> Type where [external]
```

In `Ref`, `[external]` tells the compiler that `Ref` is an opaque type which may be constructed. Otherwise, the compiler assumes a `Ref` value will never exist since it has no constructors, which may lead to code being optimized away. Rather than a constructor, an `%unsafe` function with `believe_me` is used; this prevents proof search from using it and further communicates to users they should think twice about using the functions.

```
%unsafe
unsafeRefPtr : Ref a' t -> Ptr t
unsafeRefPtr = believe_me

%unsafe
unsafePtrRef : Ptr t -> Ref a' t
unsafePtrRef = believe_me
```

ST

The underlying mechanism for scopes in this library comes from the `ST` type ([2]). It describes mutable computation within a pure language by using some type magic to limit where mutation of needed values will happen.

```

data ST : (s : Type) -> (a : Type) -> Type where
  MkSt : (s -> (a, s)) -> ST s a

data STRef : (s : Type) -> (a : Type) -> Type where
  MkRef : (s : Type) -> a -> STRef s a

newSTRef    : a -> ST s (STRef s a)
readSTRef   : STRef s a -> ST s a
writeSTRef  : STRef s a -> a -> ST s ()

runST : ({ s : Type } -> ST s a) -> a

```

ST consists of a function that takes some state *s* and returns a result *a* along with some, possibly new, state *s*. Mutable values live inside an STRef, note that it also has with an associated state type *s* but no values of *s*. The reason for this makes sense considering the `new`, `read`, and `write` functions, the *s* of STRef they operate on and the ST they produce must be the same. Meaning, an STRef created within ST *s*₁ cannot be used within ST *s*₂.

Further, consider `runST` the type of `runST` (the function which gets the result out of ST): `{ s : Type } -> ST s a` means the ST needs to work for *all* possible state types *s*. Said another way, `runST` determines what the *s* should be for the ST and consequently for all the STRefs it creates; thus an STRef for one invocation of `runST` cannot be used for another. Consider the following example:

```

failing "When unifying"
testRunSTBad : Int
testRunSTBad = runST $ readSTRef (runST $ newSTRef 2)

```

`runST $ newSTRef 2` returns a STRef `SomeS Int`, where `SomeS` is a specific type originating from `runST`. `readSTRef` then assumes this type for its output `ST SomeS Int`. But the outer `runST` needs an ST that works for *all* possible *s* while the one provided only works for `SomeS`. While STRef is able to escape the `runST` invocation it is only usable within the invocation. (footnote: what I refer to as an “in (footnote: What I refer to as an “invocation of `runST`” is called a “thread” in the literature, not to be confused with a “thread” used for concurrency),

With this understanding, a simplified version of a scope can be defined. `Lifetime` takes the place of *s* and the computation inside `Scope` accumulates pointers and a result *a*.

```

data SimpleScope : Lifetime -> Type -> Type where
  MkScope : (Unit -> (List AnyPtr, a)) -> SimpleScope s a

newRefScope : a -> SimpleScope s (Ref s a)
readRefScope : Ref s a -> SimpleScope s a

runScope : ((s : Lifetime) -> SimpleScope s a) -> a

runSubScope : (s : Lifetime) -> ((s' : Lifetime) -> SimpleScope s' a) -> a

```

This simplified formulation highlights the similarities to ST but is missing two major aspects: allocation and deallocation require IO, the *as* should somehow be related to CType. The former can quickly be addressed as it's needed for the next section, `Marshal` which addresses the latter.

```

data ScopedIO : Lifetime -> Type -> Type where
  MkScopedIO : IO (List AnyPtr, a) -> ScopedIO a' a

runScopedIO :
  HasIO io =>
  ((0 a' : Lifetime) -> ScopedIO a' a) ->

```

```
io a
```

```
-- can be ignored, these inherit properties of IO which make the computations
-- easier to compose
Functor (ScopedIO a')
Applicative (ScopedIO a')
Monad (ScopedIO a')
HasIO (ScopedIO a')
```

However, for the rest of this section the simplified version will be used for brevity. It should also be noted that, `s` was used in this example for clarity but conventionally variable names of the form `a'`, `b'`, etc. will be used for `Lifetimes`; this follows the Rust `'a` form.

Subscopes

One notable difference of `SimpleScope` from `ST` is the `runSubScope` function which creates a sub-scope from a parent scope. A desired feature is then for `readRefScope` to be able to read a reference from the parent scope `Ref s a` and return a value in the smaller scope `SimpleScope s' a`. That is, reference with a longer lifetime should be allowed in places where a lifetime with a shorter lifetime is needed (footnote: the real story is more nuanced once variance is considered [1]). As formulated, this will not work. Further, in Rust this is modeled using subtyping, a notoriously troublesome ([4], TODO: check this reference [3]), type system feature which Idris avoids. In Idris, this relationship will have to be modeled explicitly using, you guessed it, dependent types.

Firstly, `Lifetime` needs to be fully defined. It behaves similarly to `s`, so it needs to carry a `Type` that can be quantified over. However, it can be created in two ways in a “root” scope from `runScope` or as a sub-scope in `runSubScope`. The latter needs to carry its parent scope as proof that it is indeed a sub scope.

```
data Lifetime : Type where
  LRoot : (θ thr : Type) -> Lifetime
  LSub  : (θ thr : Type) -> (θ _ : Lifetime) -> Lifetime
```

This models the creation of the subscope, but it is not enough to describe the relationship as a constraint in the type level. That is, a type describing a proposition is needed. The proposition is that a lifetime `a'` lives as long as another lifetime `b'`.

```
data AtLeastAsLong : Lifetime -> Lifetime -> Type where
```

There two trivial cases:

- if `a'` and `b'` are actually the same lifetime, then they live as themselves
- if `a'` is a direct parent of `b'`, that is `b' = LSub _ b'` then `a'` does live as long as `b'`

```
ALALSame    : AtLeastAsLong a' a'
ALALParent  : AtLeastAsLong a' (LSub b a')
```

And one non-trivial case: the relationship is transitive, if `a'` lives as long as `b'` and `b'` lives as long as `c'`, then `a'` lives as long as `c'`. This should be implemented directly as:

```
ALALTrans   : AtLeastAsLong a' b' -> AtLeastAsLong b' c' -> AtLeastAsLong a c
```

But there's a bug in the compiler (TODO: minimize and open issue for this) that results in an infinite loop when doing proof search. To work around this, `ALALTrans` is defined as:

```
ALALTrans   : AtLeastAsLong a' (LSub b a') -> AtLeastAsLong (LSub b a') c' ->
AtLeastAsLong a' c'
```

An improved version of `readRefScope` can then use an auto-implicit `AtLeastAsLong` as proof that the lifetime of the reference lives as long as the scope it's computing.

```
readRefScope' :
  Ref a' a ->
  { auto p : AtLeastAsLong a' b' } ->
  SimpleScope b' a
```

Lastly, an `AtLeastAsLong` needs to be provided by `runSubScope` to the function that generates the subscope.

```
runSubScope' :
  (0 a' : Lifetime) ->
  ((0 b' : Lifetime) -> { p : AtLeastAsLong a' b' } -> SimpleScope b' a) ->
  SimpleScope a' a
```

And similarly for `ScopedIO`.

```
runSubScopedIO :
  (0 a' : Lifetime) ->
  (
    (0 b' : Lifetime) ->
    { 0 p : AtLeastAsLong a' b' } ->
    ScopedIO b' a
  ) ->
  ScopedIO a' a
```

—

Marshalling and unmarshalling

In the previous section, read and write operations were given without justification. In ST, the values can simply live in the host language. However, for the purposes of this library reads and writes face two issues:

- access to fine grained, assembly style memory loads and stores
- encoding and decoding values from the host language to a C-like representation

Together this is the problem of marshaling and unmarshalling values between the host language and C. Marshaling refers to the direction from the host language into C, while unmarshalling is the reverse.

Fine grained memory access

Firstly, compiler primitives for fine grained memory access needs to be implemented in the back end. The primitives described here are intended to be minimal for ease of implementation. They are, functions for reading and writing register-sized values to memory; a single function for pointer arithmetic; and functions for reading the raw bits of floating point values.

The pointer read and write functions have the following form with values 8, 16, 32, and 64 for `N`. (footnote: pointer reads are defined here to have side effects, this was chosen arbitrarily, but one may argue the opposite “it’s only reading not mutating” but equally true is that “invalid pointer reads trigger interrupts”)

```
prim_ptrWriteN : AnyPtr -> BitsN -> PrimIO Int64
prim_ptrReadN  : AnyPtr -> PrimIO BitsN
```

Next, pointer arithmetic is simply addition. (footnote: there may be other formulations of this eg. functions that directly convert a pointer to an integer). Notably, it explicitly uses `Int64`; this can safely be optimized away into smaller bit-widths on smaller architectures.

```
%foreign "scheme: (lambda (a b) (+ a b))"
prim_offsetPtr : AnyPtr -> Int64 -> AnyPtr
```

Lastly, functions for getting the underlying bits out of floats and pointers. Note that this not a cast which truncates a single-precision float 1.1 into 1, rather it should reinterpret the bytes into the 32 bit int 1066192077.

```
prim__SingleToBits : Double -> Bits32
prim__DoubleToBits : Double -> Bits64

prim__SingleFromBits : Bits32 -> Double
prim__DoubleFromBits : Bits64 -> Double

prim__PtrToBits : AnyPtr -> Bits64
prim__PtrToBits = believe_me
```

The Marshal type

These primitives don't provide many safety guarantees and don't compose very well. The lifetimes above can be used to create a safe and composable Marshal type:

```
data Marshal : Lifetime -> ity -> cty -> Type where
  MkMarshal : { auto repr : CType cty } -> (ity -> Ptr cty -> IO Int64) -> Marshal a'
ity cty
```

A `Marshal a' ity cty` holds a function that can marshal the Idris type `ity` (which can depend on `a'`, that is have a lifetime of `a'`) into the C type `cty`. One might expect `repr : CType cty` to be necessary for `Marshal` but pushing the constraint to the constructor makes the type easier to work with. The single constructor takes the `repr` proof instead, and the actual function for doing the marshalling, `ity -> Ptr cty -> IO Int`. Another unexpected detail is that the marshaling function takes a `Ptr` instead of a `Ref` even though the lifetime is available. This is because an implementation of `Marshal` would immediately use `unsafeRefPtr` to get a `Ptr` anyways so not much protection is happening.

The use of the lifetime on `Marshal` has two purposes. On the caller side, `newRef`, `writeRef`, can tie the `Marshal` lifetime to the output `ScopedIO` and `Ref` lifetimes. On the implementation side, `Marshal` for `Ref` can have their lifetimes tied. Without the lifetime a `Marshal` instance for `Ref` would work for a `ScopedIO` of any lifetime.

```
writeRef :
  { auto marshal : Marshal a' ity cty } ->
  ity ->
  Ref a' cty ->
  ScopedIO a' Int64
writeRef @{ MkMarshal f } x ptr = liftIO $ f x (unsafeRefPtr ptr)
```

```
newRef : ity -> { auto marshal : Marshal a' ity cty } -> ScopedIO a' (Ref a' cty)
newRef init = do
  let MkMarshal { repr } _ = marshal
  ptr <- liftIO $ alloc repr
  _ <- writeRef { marshal } init (unsafePtrRef ptr)
  pure $ unsafePtrRef ptr
```

Implementing a `Marshal` is straightforward for base types, the code below has some noise to appease the type system, but it boils down to writing to a pointer (`prim_ptrWrite`).

```
export
%hint
marshalInt : Marshal a' Int Int
marshalInt = MkMarshal $ \x, ptr =>
  primIO $ prim_ptrWrite32 (prim_forgetPtr ptr) (cast x)
```

And, through the magic of proof search, `newRef` can be called without needing to mention `marshalInt`.

```
testNewRefInt : ScopedIO a' (Ref a' Int)
testNewRefInt = newRef (the Int 25)
```

And here's how the lifetime on `Marshal` is used when marshaling `Ref` into a `Ptr`.

```
export
%hint
marshalRef : { repr : CType t } -> Marshal a' (Ref a' t) (Ptr t)
marshalRef = MkMarshal $ \x, ptr =>
    primIO $ prim_ptrWrite64
        (prim_forgetPtr ptr)
        (prim_PtrToBits $ prim_forgetPtr $ unsafeRefPtr x)
```

—

Marshalling into a `Struct` is more complicated. Firstly, there's the question of which Idris type should represent a struct. Secondly, alignment and padding of fields within the struct needs to be calculated, as with `sizeof` the details are omitted from this document.

The Idris type I've chosen to marshal into structs is tuples. (footnote: `HList` is another possible choice but, in practice, proof search struggles to find `Marshal` implementations as too many other types are defined with the `Nil` and `::` constructors)

In pseudo-code the `Marshal` instances for structs should look like:

```
Marshal a' (ix, iy, iz, ...) (Struct _ [(_, cx), (_, cy), (_, cz), ...])
```

However, there are a few issues. Firstly, 1-tuples do not exist because Idris desugars them into nested `Pairs` which have exactly two elements. (footnote:

```
(1, True, "hello") == MkPair 1 (MkPair True "hello")
```

) Secondly, each `ix` and corresponding `cx` needs its own `Marshal` instance. Implementing each of these to some finite limit is acceptable in lesser language such as Rust, but a beautiful, dependently typed language like Idris allows us to avoid this by using recursion. The base case is defined such that a `Marshal` for a plain value can be used to construct a `Marshal` for a `Struct` with one field.

```
%hint
marshalStructBase :
    { 0 ia : Type } ->
    { auto repr : CType ca } ->
    Marshal a' ia ca ->
    Marshal a' ia (Struct name [(f, ca)])
```

The recursive case combines a `Marshal` for a single value with a `Marshal` for a `Struct` and creates a `Marshal` for a `Struct` with the additional field.

```
%hint
marshalStructRec :
    { auto repr : CType ca } ->
    Marshal a' ia ca ->
    { auto reprs : All (CType . Builtin.snd) cb } ->
    Marshal a' ib (Struct name cb) ->
    Marshal a' (Pair ia ib) (Struct name ((field, ca)::cb))
```


The following examples further illustrate the power of proof search. Instances of `Marshal` can be built up automatically for only the cost of writing `%hint`. Other languages might require metaprogramming, such as macros, which have their own learning curve. However, with proof search, the programmer simply writes Idris code.

```
testNewRefStruct1 : ScopedIO a' (Ref a' (Struct "hello" [("f", Int)]))
testNewRefStruct1 = newRef (the Int 25)

testNewRefStruct2 : ScopedIO a' (Ref a' (Struct "hello" [("f", Int), ("g", Int)]))
testNewRefStruct2 = newRef ((the Int 25), (the Int 50))

testNewRefStruct3 : ScopedIO a' (Ref a'
  (Struct "hello"
    [("f", Int), ("g", Ptr Int)]))
testNewRefStruct3 = newRef ((the Int 25), !testNewRefInt)

testNewRefStruct4 : ScopedIO a' (Ref a'
  (Struct "hello"
    [ ("f", Int)
    , ( "g" , Struct "world"
        [ ("h", Int)
        , ("i", Ptr Int)
        ])
    ])
  )
testNewRefStruct4 = newRef ((the Int 25), (the Int 50, !testNewRefInt))
```

The Unmarshal type

Unmarshalling follows the same pattern, however compound types like `Struct`, cannot be unmarshalled. Instead, the API allows “projection” of references.

```
data Unmarshal : Lifetime -> ity -> cty -> Type where
  MkUnmarshal : { auto repr : CType cty } -> (Ptr cty -> IO ity) -> Unmarshal a' ity
  cty

readRef :
  { auto unmarshal : Unmarshal a' ity cty } ->
  { auto 0 p : AtLeastAsLong a' b' } ->
  Ref a' cty ->
  ScopedIO b' ity
readRef @{ MkUnmarshal f } ref = liftIO $ f (unsafeRefPtr ref)

%hint
unmarshalInt : Unmarshal a' Int Int
unmarshalInt = MkUnmarshal $ \ptr =>
  map cast $ primIO $ prim_ptrRead32 (prim_forgetPtr ptr)
```

Reference projection means a reference to a struct can be turned into a reference, of the same lifetime, to one of the struct’s fields. Once again, this requires a proposition type to guarantee that the value of the field does exist.

```
data Field : String -> List (String, Type) -> Type where
  First : Field name ((name, ty)::fs)
  Later : Field name fs -> Field name (f::fs)

0 FieldType : Field fname fs -> Type
FieldType (First { ty }) = ty
```

```
FieldType (Later l) = FieldType l
```

```
getField :  
  { auto reprs : All (CType . Builtin.snd) fields } ->  
  Ref a' (Struct name fields) ->  
  (f : String) ->  
  { auto F : Field fname fields } ->  
  Ref a' (FieldType F)
```

Examples

```
testScope1 : IO Int  
testScope1 = runScopedIO $ \a' => do  
  x <- newRef { cty = Int } $ the Int 10  
  runSubScopedIO a' $ \b' => do  
    readRef x
```

```
failing "Can't solve constraint"  
testScope2 : IO Int  
testScope2 = runScopedIO $ \a' => do  
  x <- runSubScopedIO a' $ \b' => do  
    newRef { cty = Int } $ the Int 10  
    readRef x
```

Discussion and caveats

womp womp

References

- [1] Subtyping and Variance - The Rustonomicon. Retrieved July 12, 2026 from <https://doc.rust-lang.org/nomicon/subtyping.html>
- [2] L Peyton. Lazy Functional State Threads.
- [3] Benjamin C. Pierce. 1992. Bounded Quantification Is Undecidable. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages - POPL '92*, 1992. ACM Press, Albuquerque, New Mexico, United States, 305–315. <https://doi.org/10.1145/143165.143228>
- [4] Dennis M. Volpano and Geoffrey S. Smith. 1991. On the Complexity of ML Typability with Overloading. *Functional Programming Languages and Computer Architecture* 523, 15–28. https://doi.org/10.1007/3540543961_2